Text-to-Cypher: An Agentic Approach

Nicolò Resta[1]

[1] *Department of Computer Science, University of Bari Aldo Moro, Via E. Orabona, 4, 70125 Bari, Italy*

Abstract

d

Keywords

LaTeX class, paper template, paper formatting, CEUR-WS

1. Introduction

For decades, the problem of bridging the gap between natural language and formal languages had always been a significant challenge, both for humans and software agents, in the field of computer science.

One instance of this challenge is the task of converting user's questions into database queries, with the main goal of retrieving relevant information from *structured* data sources starting from *unstructured* user queries expressed in natural language. This class of tasks, commonly referred to as Text-to-Query, is fundamentally a challenge of semantic translation and alignment between two distinct representational paradigms: the often ambiguous nature of human language and the rigid, formal structure of query languages.

The proliferation of large-scale Knowledge Graphs (KGs) has revolutionized how structured information is stored and integrated across domains ranging from biomedicine to financial intelligence. Among these, Property Graph Databases, most notably Neo4j ones with their declarative query language Cypher, have emerged as the industry standard for representing and querying complex, interconnected data.

Text-to-Cypher, a specialized subset of the broader Text-to-Query task, focuses on translating natural language questions into Cypher queries that can be executed against Property Graph Databases. This task is particularly challenging due to the inherent complexity of graph structures to express and the way in which Cypher allow us to represent such complex graph patterns (that requires some vision capabilities and intelligence to better interpret and understand ASCII art styled cypher queries).

To solve this class of tasks, an intelligent agent must be able to perform, at least, the following sub-tasks:

1. **Relevant Concepts Identification**: extracting key *entities*, *relationships* between them and their *properties* of our interest expressed in natural language. This process is the most problematic one since it is often carried out by abstract innate human cognitive processes, guided by specific intents, which are difficult to formalize computationally and requires advanced natural language understanding with a deep world knowledge.
2. **Relevant Formalisms Discovery**: discoverying the appropriate formal constructs, of the underlying used formalism framework, that had or could be used to represent identified concepts. This step involves understanding the syntax and semantics of this symbolical framework by performing multiple reasoning and action steps. In our case, the underlying framework is represented by the Property Graph Data Model and the Cypher formal query language. In our case, the discovery process involves exploring the target graph database schema to find relevant

Woodstock'22: Symposium on the irreproducible science, June 07–11, 2022, Woodstock, NY

n.resta6@studenti.uniba.it (N. Resta)

https://github.com/ashkihotah (N. Resta)

node labels, *relationship types* and their *properties* that had or can be used to represent identified concepts.

3. **Concepts-Formalisms Linking**: establishing correspondences between identified concepts and discovered formal constructs. In our case, some instances of this sub-task can be referred to as **Entity Linking**, **Relationship Linking** and **Property Linking**.
4. **Formalisms Framework Exploitation**: leveraging the established correspondences to construct syntactically and semantically correct queries in the target formalism. In our case, this involves generating Cypher queries that accurately represent the user's natural language questions based on the linked concepts and formal constructs.

All these sub-tasks are inherently probabilistic or possibilistic in nature and require a deep understanding of natural language, general world knowledge and the target formalism, as well as the ability to reason about the relationships between concepts and their representations.

With the advent of Large Language Models (LLMs) and their agentic capabilities, new avenues had opened for addressing the inherent challenges in this task. Their proficiency in understanding natural language, reasoning over complex information and generating coherent text to perform specific actions, makes them well-suited for performing the sub-tasks outlined above.

In this paper, we propose an agentic solution to Text-to-Cypher tasks, without any kind of fine-tuning, that leverages LLMs to first find all relevant schema components in the target graph database, necessary for query construction, and then generate the final Cypher query to be executed.

Contributions Our core contributions are summarized as follows:

- We designed a zero-shot approach to tackle Text-to-Cypher tasks by providing to a *retriever* LLM agent the necessary tools and connections to explore the target graph database and gather only a pruned, strictly query-specific subgraph of the schema for query construction.
-
- We evaluated our approach on well-known benchmarks for Text-to-Cypher tasks, showing its effectiveness and potential, even across completely different knowledge graphs, for future research in this area.

2. Related Works

The current landscape of Text-to-Query research is marked by a variety of approaches that leverage different techniques and methodologies to address typical problems encountered in this class of tasks. Broadly speaking, we can categorize existing works into two main lines of research: (i) the development of benchmark datasets specifically designed for training and evaluating Text-to-Query models, and (ii) the design of general Text-to-Query solutions that can be adapted to various types of databases and query languages. For our purposes, in the first line of research, we are mainly interested in datasets focused on the Text-to-Cypher task, while in the second line of research, we will explore more general Text-to-Query solutions that can be adapted to our specific task.

2.1. Text-to-Cypher Datasets

An example include the neo4j/text2cypher-2024v1 dataset introduced by the Neo4j-Labs team [1], which brings together instances from publicly available datasets, cleaning and organizing them for smoother use. This dataset consists of 44,387 instances, with 39,554 instances in the training split and 4,833 instances in the test split where for each instance, a natural language question is paired with: its corresponding Cypher query, the entire schema of the underlying graph database and, if available, an identifier to the specific Neo4j database hosting the entire knowledge graph from which the instance was derived. The Neo4j-Labs team, in fact, provide a remote access to multiple already deployed Neo4j graph databases, freely and publicly accessible, that can be used to execute the Cypher queries present

in the dataset. This important aspect paired with the vast diversity of the included knowledge graphs (about 17 different Neo4j graph databases are used to derive the instances in the dataset), makes it well-suited for our purposes.

Another notable example is the megagonlabs/cypherbench dataset introduced by Megagon Labs [2] with 10,882 instances for training and evaluating text-to-Cypher models. To create such a benchmark, the authors first developed a novel RDF-to-Property-Graph transformation engine that converts RDF data into high quality locally deployable property knowledge graphs while preserving the original semantics. Then, they applied this engine to Wikidata, generating 11 large-scale multi-domain property graphs that captures a wide range of real-world entities and relationships. Lastly, they designed a synthetic data generation pipeline that, starting from graph patterns templates, automatically creates diverse natural language questions and their corresponding Cypher queries based on the structure and content of the generated property graph. The peculiarity of this approach lies in its focus on comprehensiveness, in graph patterns covered, and diversity, in graph schemas and use cases addressed, of the resulting benchmark dataset. However, the lack of already deployed, publicly accessible and remote Neo4j graph databases endpoints, force anyone willing to use this dataset to first locally deploy the generated property graphs before being able to execute the Cypher queries present in the dataset. To further complicate things, the configuration method of such local deployments provided by the authors requires significant hardware requirements (at least 64 GB of RAM when deploying the 7 test graphs and 128GB when deploying all 11 graphs[1]) that are out of reach for many researchers. For this reason, we decided to not use this dataset for our experiments.

2.2. General Text-to-Query Solutions

Several solutions have been proposed to tackle the general Text-to-Query translation task, primarily leveraging transformer-based architectures [3], both decoder-only and encoder-decoder models, in combination with various techniques. To address problem 4 in Section 1 and ensure the generation of **schema-compliant**, syntactically and semantically correct queries, researchers have extensively adopted **in-context learning** (ICL) combined with **prompt engineering**. These techniques exploit the few-shot and zero-shot capabilities of large language models by providing carefully crafted prompts containing *task instructions*, *few-shot demonstrations* (retrieved from the training set), and relevant contextual information such as *schema elements* to guide query generation [1, 4, 5, 2, 6].

Schema Retrieval Approaches. Regarding the inclusion of schema information in prompts, most existing works [1, 2, 4, 6] either assume that relevant schema components are already known or simply provide the entire database schema without any filtering or pruning. This approach, while straightforward, can lead to excessively large prompts that may exceed the model's maximum context length or introduce irrelevant information that degrades generation quality.

To address problems 1, 2, and 3 in Section 1, only a few works have proposed techniques for **targeted schema retrieval**. Some approaches [5, 7, 8] combine lexical matching methods (e.g., Levenshtein distance, fuzzy string matching, token-based or prefix matching) for schema element identification with semantic similarity techniques (e.g., BM25 sparse retrieval or dense embeddings from Sentence Transformers [9]) for instance retrieval.

Trade-offs in Similarity Techniques. These retrieval methods, however, present inherent trade-offs. Lexical matching excels at identifying exact matches or minor spelling variations in short terms but becomes less effective for longer phrases where semantic understanding is crucial (e.g., handling synonymy, polysemy, and other lexical-semantic relationships). Conversely, contextual dense embeddings from transformer models excel at capturing semantic similarity and scale efficiently to large databases through vector stores. However, they can be inefficient or ineffective for short, information-dense

[1]This was explicitely stated in the 3rd step of the quick start of the `README.md` file in their repo https://github.com/megagonlabs/cypherbench

terms such as node labels (`DataCenter`, `PointOfInterest`), relationship types (`PARENT_OF`, `DEPENDS_ON`), or property keys (`created_at`, `description`) which lack the rich explicit context needed to generate discriminative contextual representations useful for retrieval purposes.

Static Word Embedding (SWEs) models like FastText [10] address this limitation by providing context-free, semantically rich representations that handle out-of-vocabulary terms through subword information. However, their prohibitive memory requirements (vocabularies with millions of tokens) make them impractical for large-scale deployment.

Model2Vec [11] offers an optimal solution for this use case. It produces static word embeddings by distilling knowledge from large Sentence Transformer models through output embedding aggregation and dimensionality reduction via PCA [12], eliminating the need for the original model's heavy neural architecture. This distillation process yields compact, semantically rich representations that retain the semantic quality of their teacher models while requiring only vocabulary-level storage and enabling inference through simple vocabulary lookups and mean pooling. For our scenario, Model2Vec provides an ideal balance: it captures rich semantic information like contextual models, operates efficiently on short terms like static embeddings, yet avoids the computational overhead of transformer inference and the memory footprint of traditional SWEs.

Algorithmic vs. Agentic Architectures. Text-to-query solutions can be further categorized based on their architectural paradigm. **Algorithmic approaches**, such as Yang et al. [5], follow predefined pipelines: first performing entity linking via lexical matching at the schema level and hybrid lexical-semantic matching at the instance level, then extracting relevant schema portions for query construction. While deterministic, these approaches face several limitations: (i) they rely on strong, domain-specific assumptions that may not generalize, (ii) they lack robust disambiguation mechanisms when multiple candidates match the same concept, and (iii) they treat relationship linking as secondary to entity linking, imposing rigid dependencies that may not hold in real-world scenarios.

In contrast, **agentic approaches** [13, 14, 15], such as the framework proposed by Walter et al. [7, 8], equip LLM agents with a set of tools to iteratively address all sub-tasks 2, 3, and 4 from Section 1. These tools include pre-built indexes for lexical and semantic search, as well as direct database query capabilities for grounding and query formulation. This architecture enables dynamic tool selection based on context, allowing the agent to adapt its retrieval strategy to specific database characteristics and handle ambiguities more flexibly than rigid algorithmic pipelines. However, ensuring efficiency and effectiveness at scale, particularly for large databases or complex queries, remains an open challenge requiring careful system design and optimization.

3. Approach

In this section, we present our agentic solution for the Text-to-Cypher task. We first describe the overall system architecture, highlighting the core components and their interactions. Then, we detail the design and implementation of the specific tools that enable our agent to effectively perform all the sub-tasks defined in Section 1.

3.1. System Architecture

Our system is built upon the LangChain[2] and LangGraph[3] frameworks, which provide robust infrastructure for implementing stateful, multi-agent workflows. The architecture employs OpenAI's GPT-4.1 as the central reasoning engine, responsible for interpreting user queries and orchestrating the execution of sub-tasks necessary to generate the final Cypher query.

To enable **Relevant Formalisms Discovery**, the system incorporates a **FAISS Vector Store** [16] for semantic retrieval of schema components. This vector store indexes node labels, relationship types, and

[2] https://www.langchain.com/

[3] https://www.langchain.com/langgraph

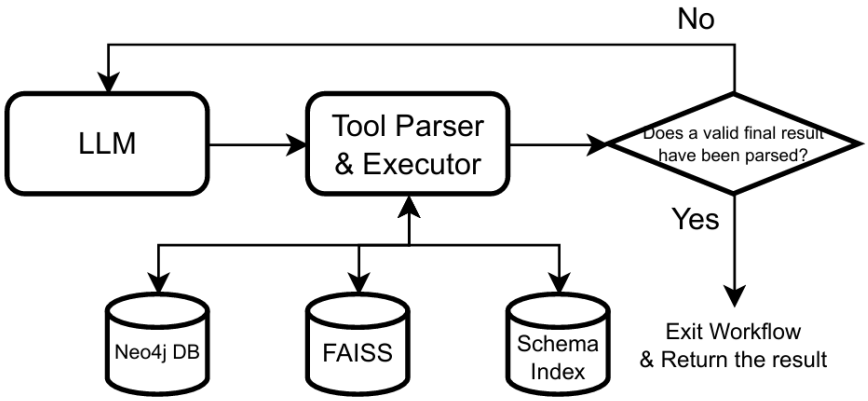

Figure 1: System Architecture

property keys from the target knowledge graph using the minishlab/potion-retrieval-32M Model2Vec embedding model, which is a finetuned version of the minishlab/potion-base-32M[4] model

To capture the structural relationships between schema components, we designed a complementary **Schema Index**—an inverted index data structure that maintains bidirectional mappings between: (i) node labels and their properties, (ii) relationship types and their properties, (iii) relationship types and their valid domain-range node label pairs, (iv) property keys and their associated node labels, and (v) property keys and their associated relationship types. This index enables efficient reverse lookups and schema traversal queries.

The system operates through a ReAct-style agentic loop [13], iteratively invoking tools and reasoning over observations until the agent either accumulates sufficient information to construct a valid Cypher query or determines that the user's question cannot be answered given the available knowledge graph.

3.2. Tool Design and Implementation

To enable the LLM agent to systematically perform the sub-tasks 2 and 3 outlined in Section 1, we designed a suite of specialized tools that facilitate efficient and effective schema exploration and information retrieval.

```
def search_for(
    components: List[Literal["nodes", "relationships", "properties"]],
    similar_to: str,
    top_k: int = 10
) -> str:
```

The `search_for` tool performs semantic search over the specified vector-indexed schema components (node labels, relationship types, and property keys) to identify all plausible candidate formalisms that could represent a given concept. The tool accepts three parameters: `components` (specifying which schema component types are included in the search space), `similar_to` (a natural language description of the concept), and `top_k` (the number of results to return). This design explicitly prioritizes *recall over precision*: the tool is intended to cast a wide net and retrieve all potentially relevant schema components, even at the cost of including some false positives. This design choice is crucial because missing a relevant schema component early in the retrieval process would be more costly to recover in subsequent steps, whereas false positives can be filtered out through later precision refinement, delegating it to subsequent tools. To maximize efficiency and minimizing latency, the tool is designed to be invoked in parallel for each independent concept within a complex query. Lastly, both

[4]Which in turn it is a Model2Vec model distilled from the baai/bge-base-en-v1.5 Sentence Transformer [17]

here and in the subsequent tools, `similar_to` and `top_k` parameters are used to limit results only to the most relevant one avoiding polluting and consuming the LLM token context window with useless information.

```
def get_properties_from(
    components: List[Literal["nodes", "relationships"]],
    of_types: List[str],
    similar_to: Optional[List[str]] = None,
    top_k: Union[int, List[int]] = 10
) -> str:
```

Once relevant node labels or relationship types have been identified, for each *i-th* element in the batch, the `get_properties_from` tool retrieves the associated properties of the schema component `components[i]` of type `of_types[i]` optionally ranked by similarity to `similar_to[i]`. This *batching-first* design philosophy is critical for two reasons: (i) it reduces the number of LLM tool calls, thereby decreasing latency and token consumption, and (ii) it encourages the agent to think holistically about multiple schema components simultaneously rather than myopically focusing on one at a time.

```
def get_domains_ranges_of(
    relationships: List[str],
    similar_to: Optional[List[str]] = None,
    order_by: List[Literal["domains", "ranges"]] = None,
    top_k: Union[int, List[int]] = 10
) -> str:
```

The `get_domains_ranges_of` tool for each relationship type `relationships[i]` retrieves valid pairs of domain (source) and range (target) node labels most similar to the `similar_to[i]` query. The `order_by[i]` parameter allows the agent to specify whether the ranking should be applied on domains or ranges. This tool is essential for understanding how different entities are interconnected within the graph, enabling the agent to reason about potential traversal paths and relationship chains necessary for constructing complex Cypher queries.

```
def get_components_with_property(
    keys: List[str],
    components: List[Literal["nodes", "relationships", "both"]],
    similar_to: Optional[List[str]] = None,
    top_k: Union[int, List[int]] = 10
) -> str:
```

While most tools follow a forward lookup pattern (from schema components to properties), `get_components_with_property` performs reverse lookup finding, for each property key `keys[i]`, which schema components `components[i]` most similar to the query `similar_to[i]` possess it. This tool is particularly valuable for disambiguating queries where discriminative properties serve as strong signals for **Relevant Formalisms Discovery** (Sub-task 2).

```
def execute_cypher_query(cypher: str) -> List[dict]:
```

The `execute_cypher_query` tool provides a flexible mechanism for the agent to validate assumptions, explore data instances, and verify the existence of specific patterns through the execution of arbitrary Cypher queries. Unlike the schema-focused tools, this tool operates at the data instance level and, therefore, it is crucial for grounding the agent's schema-level reasoning in actual data instances, enabling it to confirm hypotheses that inform the final query construction.

```
def register_retrieval_result(
    motivation: str,
    cypher_query: Optional[str] = None,
    kg_schema: Optional[KGSchema] = None
) -> RetrievalAgent.Result:
```

The `register_retrieval_result` tool allow the LLM to end the ReAct loop by summarizing the findings with a structured output containing: (i) the `motivation` explaining why the user's query can or cannot be answered with a cypher query given the discovered schema components in the provided knowledge graph, (ii) optionally the constructed `cypher_query`, and (iii) an optional `kg_schema` object that encapsulates the discovered sub-schema relevant to the user's query. This structured output is essential for downstream components that may need to further process or execute the generated Cypher query.

This carefully orchestrated tool ecosystem enables the LLM agent to systematically navigate the complex space of schema exploration, moving from broad conceptual understanding to precise formal query construction while maintaining both comprehensiveness and efficiency.

4. Experiments

In this section, we present the experimental setup and results of our approach. We evaluated our method only on instances of the neo4j/text2cypher-2024v1 test set that have an already deployed Neo4j database available on Neo4j Sandbox. This reduced the number of test instances from 4,833 to 2471, but it was strictly necessary to, at least, execute our agentic approach against the actual database used for the generation of samples in the test set. Furthermore, we used already computed zero-shot and finetuned results reported in the original paper [1] as several baselines to compare our approach with their one.

4.1. Evaluation Metrics

We evaluate our text-to-Cypher approach using a combination of runtime-based and structural metrics, following the evaluation protocols established in prior work [1, 2]. These metrics collectively assess both the syntactic correctness and semantic equivalence of generated Cypher queries.

Execution Accuracy (EA). This metric measures whether a generated Cypher query executes successfully on the target Neo4j database without raising syntax or runtime errors. A query is considered valid if it can be executed within a specified timeout (120 seconds in our experiments) without encountering exceptions such as `CypherSyntaxError`, `DatabaseError`, `CypherTypeError`, or `ClientError`. Execution Accuracy provides a fundamental threshold for query quality, as non-executable queries offer no practical value regardless of their structural similarity to ground truth.

Exact Match (EM). The Exact Match metric represents the most stringent evaluation criterion, requiring character-level equivalence between the predicted and ground truth Cypher queries. While highly precise, this metric is known to be overly conservative, as semantically equivalent queries may differ in syntactic representation (e.g., variable naming, clause ordering, or equivalent formulations). Nevertheless, EM serves as an important upper bound for perfect generation accuracy.

Result Similarity (RS). To assess semantic equivalence at the data level, we compute the Jaccard similarity between the result sets returned by the predicted and ground truth queries. Specifically, we represent each query result as a list of dictionaries, where each dictionary corresponds to a row in the output. The similarity computation accounts for row ordering when either query contains an `ORDER BY` clause; otherwise, we apply a greedy alignment algorithm that pairs rows from both result sets to maximize overall similarity. For each aligned pair of rows, we convert values to hashable types and compute their Jaccard index. The final Result Similarity is the Jaccard coefficient over all (row index, value) pairs across both result sets. This metric captures whether queries retrieve the same data, even when their structural implementations differ. Following [1], a value of RS ≥ 0.5 is considered a successful match.

Provenance Subgraph Jaccard Similarity (PSJS). Introduced by [2], this metric evaluates the structural overlap of the graph patterns matched by two queries, rather than their final result sets. The provenance subgraph of a query consists of all nodes and relationships traversed during pattern matching, regardless of what is ultimately returned. For each query, we extract the `MATCH` and `WHERE` clauses, augment unlabeled nodes and relationships with temporary variables, and construct a Cypher query that returns the element IDs of all matched nodes. The PSJS is then computed as the Jaccard coefficient between the sets of element IDs from the predicted and ground truth provenance subgraphs:

$$\text{PSJS} = \frac{|PS_{\text{pred}} \cap PS_{\text{target}}|}{|PS_{\text{pred}} \cup PS_{\text{target}}|}$$

where PS_{pred} and PS_{target} denote the provenance subgraphs of the predicted and target queries, respectively. This metric is particularly valuable for evaluating whether a model correctly understands the graph structure to be queried, even if the final projection or aggregation differs from the ground truth. As with Result Similarity, we adopt the threshold of PSJS ≥ 0.5 as a success criterion following [2].

Together, these metrics provide a multi-faceted evaluation framework that captures syntactic validity (EA, EM), semantic data equivalence (RS), and structural query understanding (PSJS), enabling comprehensive assessment of text-to-Cypher generation quality.

4.2. Results

References

[1] M. G. Ozsoy, L. Messallem, J. Besga, G. Minneci, Text2cypher: Bridging natural language and graph databases, 2024. URL: https://arxiv.org/abs/2412.10064. arXiv:2412.10064.

[2] Y. Feng, S. Papicchio, S. Rahman, Cypherbench: Towards precise retrieval over full-scale modern knowledge graphs in the llm era, arXiv preprint arXiv:2412.18702 (2024).

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, 2023. URL: https://arxiv.org/abs/1706.03762. arXiv:1706.03762.

[4] J. Longwell, M. Ali Akbar Alavi, F. Zarrinkalam, F. Ensan, Triple augmented generative language models for sparql query generation from natural language questions, in: Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region, SIGIR-AP 2024, Association for Computing Machinery, New York, NY, USA, 2024, p. 269–273. URL: https://doi.org/10.1145/3673791.3698426. doi:10.1145/3673791.3698426.

[5] C. Yang, C. Li, X. Hu, H. Yu, J. Lu, Enhancing knowledge graph interactions: A comprehensive text-to-cypher pipeline with large language models, Information Processing & Management 63 (2026) 104280. URL: https://www.sciencedirect.com/science/article/pii/S0306457325002213. doi:https://doi.org/10.1016/j.ipm.2025.104280.

[6] J. D'Abramo, A. Zugarini, P. Torroni, Investigating large language models for text-to-SPARQL generation, in: W. Shi, W. Yu, A. Asai, M. Jiang, G. Durrett, H. Hajishirzi, L. Zettlemoyer (Eds.), Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing, Association for Computational Linguistics, Albuquerque, New Mexico, USA, 2025, pp. 66–80. URL: https://aclanthology.org/2025.knowledgenlp-1.5/. doi:10.18653/v1/2025.knowledgenlp-1.5.

[7] S. Walter, H. Bast, GRASP: generic reasoning and SPARQL generation across knowledge graphs, in: ISWC (1), volume 16140 of *Lecture Notes in Computer Science*, Springer, 2025, pp. 271–289.

[8] S. Walter, H. Bast, Knowledge graph entity linking via interactive reasoning and exploration with GRASP, in: OM 2025 (Ontology Matching Workshop), CEUR Workshop Proceedings, CEUR-WS.org, 2025. To appear.

[9] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, 2019. URL: https://arxiv.org/abs/1908.10084. arXiv:1908.10084.

[10] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, Enriching word vectors with subword information, 2017. URL: https://arxiv.org/abs/1607.04606. arXiv:1607.04606.

[11] S. Tulkens, T. van Dongen, Model2vec: Fast state-of-the-art static embeddings, 2024. URL: https://github.com/MinishLab/model2vec. doi:10.5281/zenodo.17270888.

[12] J. Shlens, A tutorial on principal component analysis, 2014. URL: https://arxiv.org/abs/1404.1100. arXiv:1404.1100.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, 2023. URL: https://arxiv.org/abs/2210.03629. arXiv:2210.03629.

[14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, 2023. URL: https://arxiv.org/abs/2302.04761. arXiv:2302.04761.

[15] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez, Gorilla: Large language model connected with massive apis, 2023. URL: https://arxiv.org/abs/2305.15334. arXiv:2305.15334.

[16] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, H. Jégou, The faiss library, 2025. URL: https://arxiv.org/abs/2401.08281. arXiv:2401.08281.

[17] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, C-pack: Packaged resources to advance general chinese embedding, 2023. arXiv:2309.07597.